MATH 564 Friday Sep 19

What are the drawbacks of quasi-Newton methods?

- High memory usage – storing $n \times n$ matrix H
- Typically needs $\sim n$ iterations to start good progress
- Relies on Line Search
- The BFGS/SR1 Ite. update is computationally "expensive".

Today we consider quasi-Newton methods that behave well for large n. That is, we cannot store H and we need efficient q-N update calculations. It will turn out that we can also address the second bullet above!

We can make some progress by realizing that while H may be dense (more than a few nonzero entries) it is of low rank. Recall BFGS is a rank 2 update and SR1 is a rank one update. So when $k \ll n$, H_k is still low rank.

Example:

$$M = \begin{pmatrix} 9 & -3 & 6 & 0 & 3 \\ -3 & 1 & -2 & 0 & -1 \\ 6 & -2 & 4 & 0 & 2 \\ 0 & 0 & 0 & 0 & 0 \\ 3 & -1 & 2 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 3 \\ -1 \\ 2 \\ 0 \\ 1 \end{pmatrix} \begin{pmatrix} 3 & -1 & 2 & 0 & 1 \end{pmatrix} = uu^T$$

We can store $5 \times 5 = 25$ entries of M or the 5 entries of u. Then any computation using M is efficient:

$$Mx = uu^T x = (u^T x)\, u$$

inner product

scalar/vector multiply

Ok. But the big question is whether or not any matrix of low rank can be written as a sum of rank one matrices.

Consider $m \times n$ real matrix M. A singular value decomposition of M is

$$M = \underbrace{U}_{\substack{m\times m \\ \text{orthogonal}}} \underbrace{\Sigma}_{m\times n \text{ diagonal}} \underbrace{V^T}_{\substack{n\times n \\ \text{orthogonal}}}$$

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_{\min(m,n)} \geq 0$$

Now choose an arbitrary $x \in \mathbb{R}^n$ and consider Mx. The goal is to find a new representation of M based on the SVD. Consider the case $m < n$.

$$Mx = U \Sigma V^T x$$

$$= \begin{pmatrix} | & & | \\ u_1 & \cdots & u_m \\ | & & | \end{pmatrix} \left(\begin{array}{cccc|c} \sigma_1 & & & & \\ & \sigma_2 & & 0 & \\ & & \ddots & & 0 \\ 0 & & & \sigma_m & \end{array}\right) \begin{pmatrix} - & v_1^T & - \\ - & v_2^T & - \\ & \vdots & \\ - & v_n^T & - \end{pmatrix} \begin{pmatrix} | \\ x \\ | \end{pmatrix}$$

$$= \begin{pmatrix} | & & | \\ u_1 & \cdots & u_m \\ | & & | \end{pmatrix} \left(\begin{array}{cccc|c} \sigma_1 & & & & \\ & \sigma_2 & & 0 & \\ & & \ddots & & 0 \\ 0 & & & \sigma_m & \end{array}\right) \begin{pmatrix} v_1^T x \\ v_2^T x \\ \vdots \\ v_n^T x \end{pmatrix}$$

$$= \begin{pmatrix} | & & | \\ u_1 & \cdots & u_m \\ | & & | \end{pmatrix} \begin{pmatrix} \sigma_1 v_1^T x \\ \sigma_2 v_2^T x \\ \vdots \\ \sigma_m v_m^T x \end{pmatrix}$$

$$= \sigma_1 u_1 v_1^T x + \sigma_2 u_2 v_2^T x + \cdots + \sigma_m u_m v_m^T x$$

$$= \left(\sum_{k=1}^{m} \sigma_k u_k v_k^T \right) x$$

Because x was arbitrary:

$$M = \sum_{k=1}^{m} \sigma_k u_k v_k^T.$$

That is, M is a linear combination of rank one matrices $u_k v_k^T$. Suppose $\text{rank}(M) = r$. Then $\sigma_{r+1}, \dots, \sigma_m = 0$ and

$$M = \sum_{k=1}^{r} \sigma_k u_k v_k^T,$$

the sum of r rank 1 matrices.

$$\boxed{\text{Every rank } r \text{ matrix can be written as the sum of } r \text{ rank one matrices.}}$$

The storage fraction for $m \times n$ rank r matrix M is

$$\text{s.f.} = \frac{rm + rn}{mn}$$

← values in r u_k's and r v_k's

← values in M

If M is square: $(m = n)$

$$\text{s.f.} = \frac{2r}{n}$$

As long as $r \ll m$ and $r \ll n$ then rank 1 decomposition is efficient.

If M is symmetric $(u_k = v_k)$

$$\text{s.f.} = \frac{r}{n}$$

Also notice the efficient computation of Mx:

$$Mx = \left(\sum_{k=1}^{r} \sigma_k u_k v_k^T \right) x = \sum_{k=1}^{r} \sigma_k (v_k^T x) u_k$$

inner product

scalar/vector product

BUT (!) The BFGS update is recursive, not additive. Let's look at it:

we have s_k, y_k, $\rho_k \equiv (y_k^T s_k)^{-1}$, $V_k \equiv I - \rho_k s_k s_k^T$

$$H_0 = H_0$$

$$H_1 = V_0^T H_0 V_0 + \rho_0 s_0 s_0^T$$

$$H_2 = V_1^T H_1 V_1 + \rho_1 s_1 s_1^T$$

$$= V_1^T \left[V_0^T H_0 V_0 + \rho_0 s_0 s_0^T \right] V_1 + \rho_1 s_1 s_1^T$$

$$= V_1^T V_0^T H_0 V_0 V_1 + \rho_0 V_1^T s_0 s_0^T V_1 + \rho_1 s_1 s_1^T$$

$$H_3 = V_2^T H_2 V_2 + \rho_2 s_2 s_2^T$$

$$= V_2^T V_1^T V_0^T H_0 V_0 V_1 V_2 + \rho_0 V_2^T V_1^T s_0 s_0^T V_1 V_2 + \rho_1 V_2^T s_1 s_1^T V_2 + \rho_2 s_2 s_2^T$$

Generalize to consider m previous iterations from the current iteration k:

$$H_k = (V_{k-1}^T \cdots V_{k-m}^T) H_{k-m} (V_{k-m} \cdots V_{k-1})$$
$$+ \sum_{j=k-m}^{k-2} (V_{k-1}^T \cdots V_{j+1}^T) s_j s_j^T (V_{j+1} \cdots V_{k-1})$$
$$+ \rho_{k-1} s_{k-1} s_{k-1}^T$$

This representation requires storage of H_{k-m}, $s_{k-m} \cdots s_{k-1}$, $y_{k-m} \cdots y_{k-1}$: 2m vectors in $\mathbb{R}^n$ and a hopefully sparse H_{k-m}. We can instead let H_{k-m} be the current initial Hessian inverse approximation H_k^0. (very beneficial)

We never need to actually compute H_k. What we need is to compute the q-N descent direction $p = H_k(-\nabla f_k)$.

Here is a clever and efficient method for computing $p = Hx$.

```
For i = k-1 to k-m
    α_i = ρ_i s_i^T x
    x ← x - α_i y_i
end
p = H_k^0 x
For i = k-m to k-1
    β = ρ_i y_i^T p
    p ← p + s_i (α_i + β)
end
```

Again, we must have

(1) H_k^0 sparse

(2) $m \ll n$

-icient

Suggested initialization : $H_k^0 = \dfrac{s_{k-1}^T y_{k-1}}{y_{k-1}^T y_{k-1}} I$

This initialization is crucial. It starts the information in H_k with the most <u>local</u> and <u>recent</u> secant information. This is likely a key reason why this partial Hessian inverse update can outperform BFGS, even for $m \ll n$.

L-BFGS Algorithm

Given : x_0
$m > 0$ integer
BFGS parameters

small values of m tend to work extremely well, say $m \sim 10$ for problems with $n \sim 1000$.

Actual results are problem dependent but the amazing thing is that small m just works well.

Set: $k \leftarrow 1$

Until some stopping criterion is met

If $k = 1$

$H_k^0 \leftarrow |f(x_0)| \, I_n$

else

$$H_k^0 \leftarrow \frac{s_{k-1}^T y_{k-1}}{y_{k-1}^T y_{k-1}} I_n$$

end

compute $p_k = H_k(-\nabla f_k)$ by $\square$ algorithm

perform strong wolfe line search

If $k > m$

discard s_{k-m} and y_{k-m}

save s_k and y_k

end

$k \leftarrow k+1$

end

- All convergence results for BFGS are retained!

- Similar algorithms exist for other q-N methods